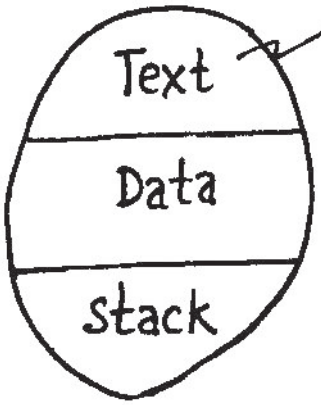
PROCESS SEGMENTS

A Process



Brain : IQ of process
does not grow

impure programs (like virus)
modify text segment:
cannot survive under UNIX.
(Text: executably only)

stack: contains stack frames of nested invokations.

Each stack frame : one process invokation
instantiation

Programming styles
- iterative (limited stack needed)
- recursive (explosive growth of stack)

Data segment

original : globals & statics
but can grow & shrink: @ programmer's choice

Growth
- involuntary (stack)
- voluntary (data)

PROCESS SEGMENTS

Questions:

(1) Concurrent growth of data segments of several processes : How does UNIX manage ?

(2) How UNIX manages stack growth of processes

(3) How data segment growth managed by the programmer ?

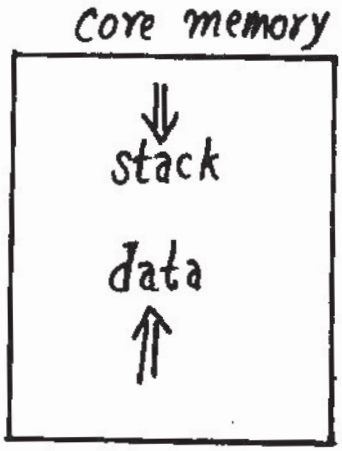
MEMORY GROWTHS

Memory growths are always to consecutive bytes.

Ex: Stack contains locals ⇒ consecutive bytes.

stack always grows downwards.

Make data segment grow upwards.



catch: under UNIX, each process has a data segment and a stack segment.

i.e. 2*N segments: all growing concurrently

analogy: all students growing fatter !

DOS MEMORY MAP

8086 : has segmented architecture

ip, sp, ax, bx, .. : 16 bits registers

cs, ds, ss, es : 16 bits segment registers

An access:

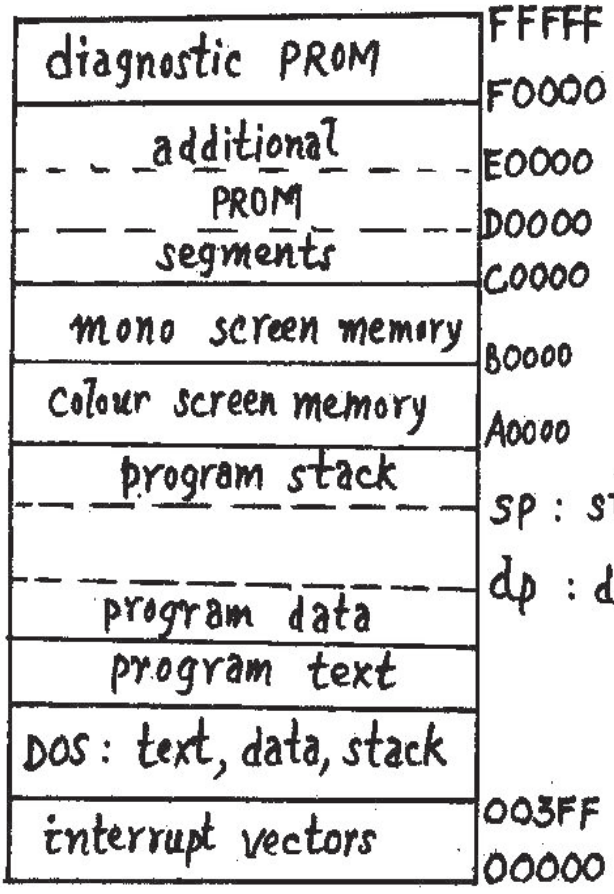
16 bit offset

16 bit segment | 0000

Add: 20 bit Address

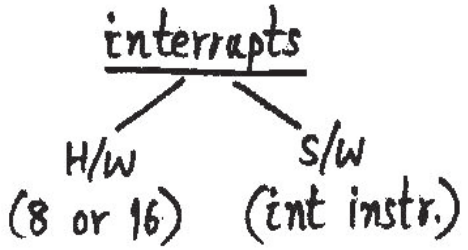
CPU address space = 1 MB.

i.e. 00000 to FFFFF



6 segments = 384 KB

Balance = 640 KB



DOS MEMORY MANAGEMENT

NOTE: data line moved up for allocation of memory.
stack line moved down for allocation of stack.

They must not intersect & cross oven.

Data growth : (ex: malloc invokation)

can check for gap & return NULL if sufficient free area not available

stack growth : is involuntary.
(OS not in the picture)

procedure preamble:

```
push.a  fp
move.a  sp, fp
sub.a   N, sp      ; N bytes : size of locals
                   ; stack is growing here.
```

compiler helps : make the involuntary growth voluntary.

```
push.a  fp
move.a  sp, fp
move.i  N, accm
call    check_stack   : or __chk_stack
sub.a   N, sp
....
```

DOS memory management

check_stack :

simply returns if sufficient gap available
else prints
** stack Overflow Error **
and returns to DOS (process killed !)

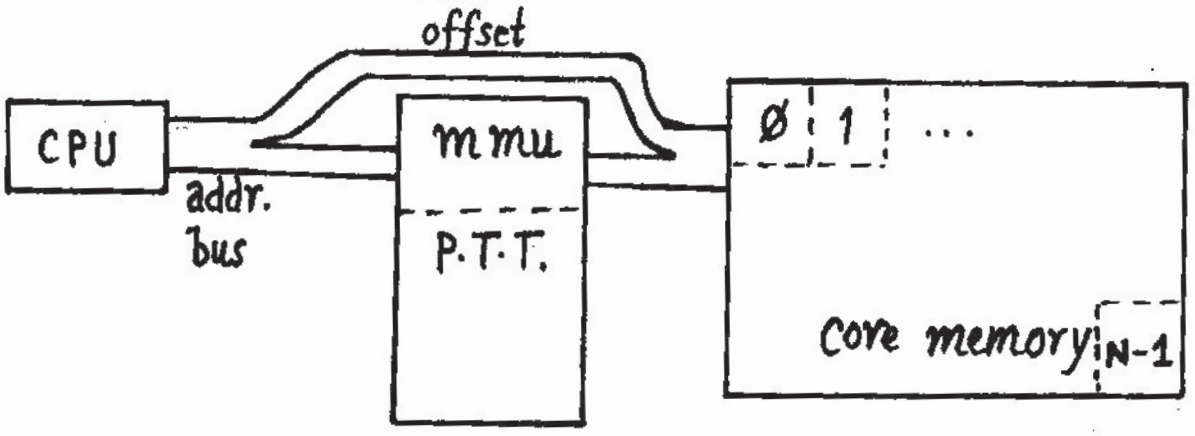
analogy: walk out of the room without banging nose against wall.
check_stack : like a tailer

Check_stack : takes exec. time (few μsec.)
(like insurance premium)

analogy: walk in a foot ball field. Ex: OS itself.

Use: cc -K while compiling.

UNIX stacks



UNIX stacks

A Process

on the moon !

: : => huge logical gap.

Sample Translation Table

0	243
1	20
40,000	

Next data : Page 2
Next stack: Page 39,999

globals, statics : birth weight

fixed size

UNIX: would like to drop the check stack premium to under μsec.

Also: UNIX can print memories (like Govt. prints money)

hence: no stack overflow at all.

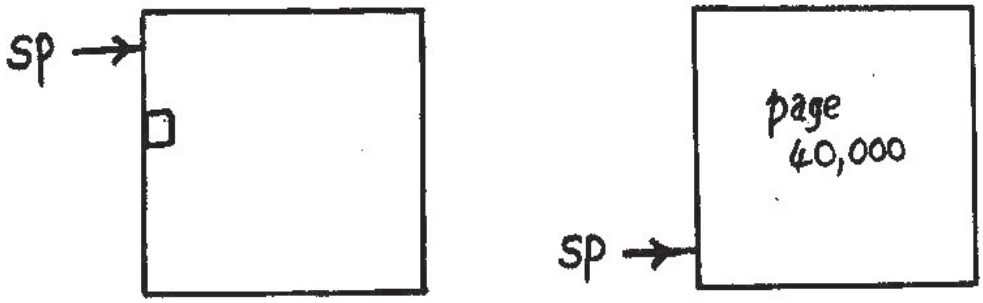
Stack Probe

preamble

```
push.a  fp
move.a  sp, fp
tst.b   (sp - N)    : a mem. access
sub.a   N, sp
```

UNIX stacks

Stack Probe : like the blindman's walking stick.



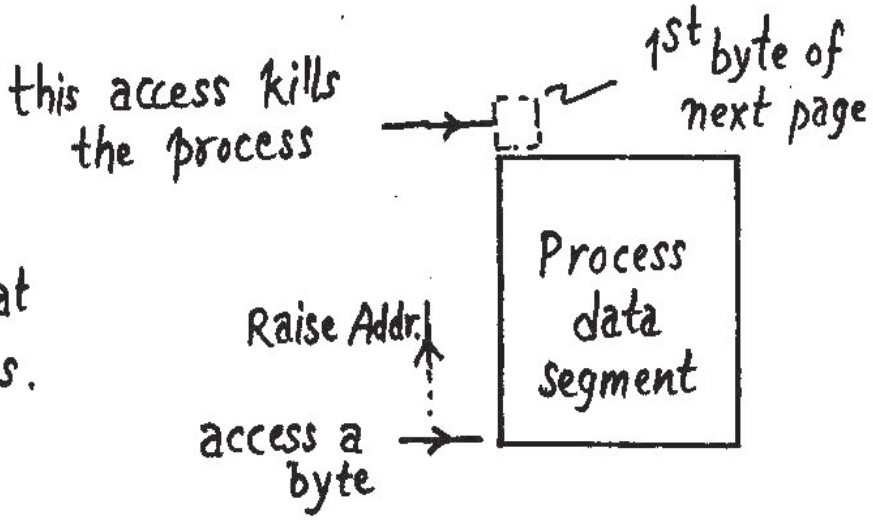
when memory insufficient,

mmu generates segmentation violation interrupt.

Interrupt service : if instruction is "tst.b",
gives one more stack page
(entry: 39,999 → 320)

DATA MANAGEMENT

voluntary growth

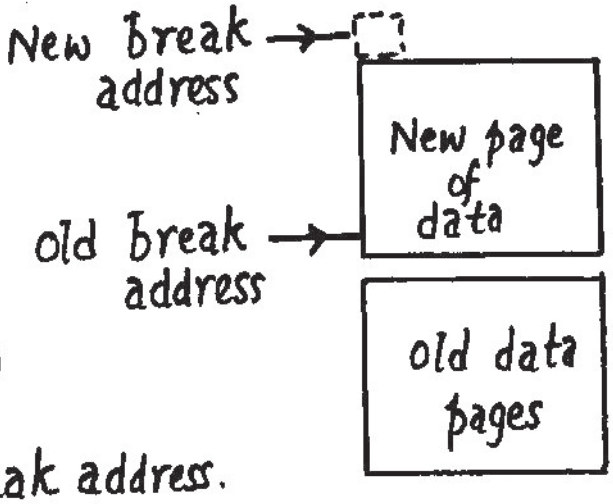


break address

min. address that
kills the process.

data management

add a page to process data:



Allocate memory
⇒ break address raised.
(by multiple of page size)

Free memory ⇒ lower break address.

brk(2), sbrk(2) : system calls

brk (char * new_break) is invokation.

templates: char * brk (char * new_break)
↳ old break address

char * sbrk (int distance);
↳ old brk address
↳ +ve or -ve

sbrk similar to malloc : malloc(100)
also returns char *

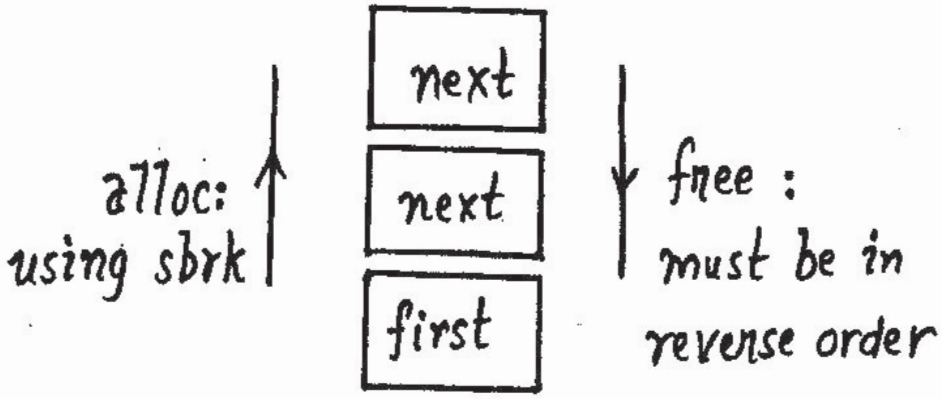
NOTE: Previous break address is address of the 1st byte of new memory allocated.

data management

Problems

(1) System call overheads
for small growths of data segment.
(sbrk : like wholesaler)

(2) many allocations
& freeing.
First memory freed : last memory allocated.



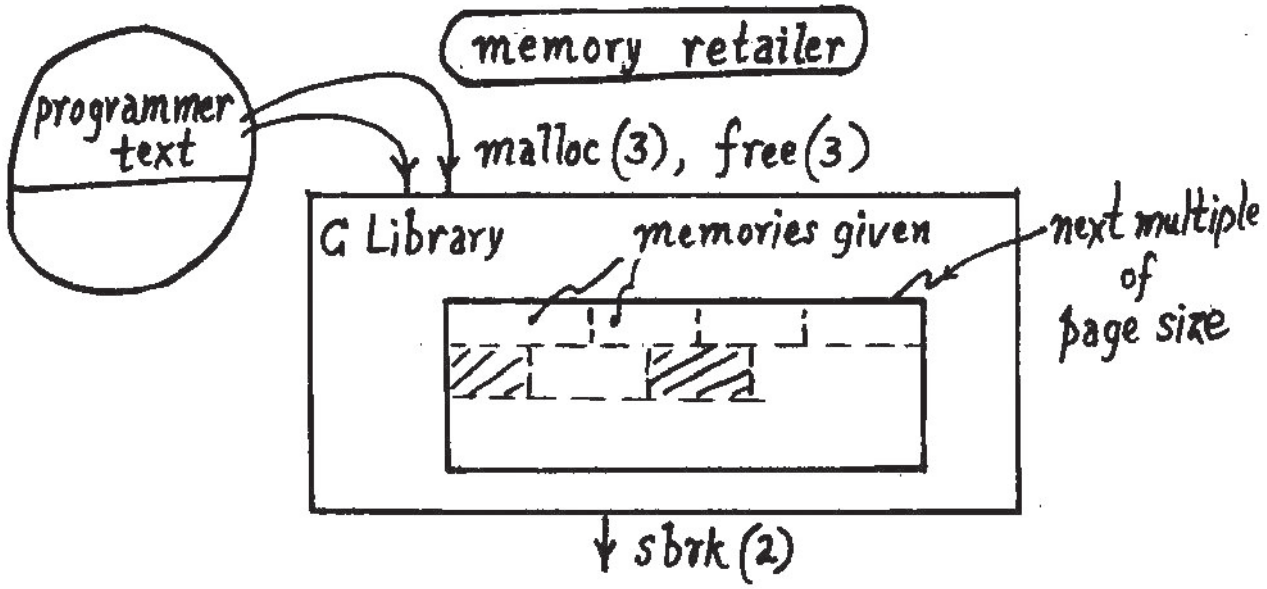
i.e. data segment behaves like stack.
this is not desirable.

We need retailer of memory like retailer of rice.

difference: start business with no rice.

malloc, free : retailers of memory.

malloc : allocates next higher # of pages
using sbrk(2).



Retailer: needs to maintain occupied & free areas.

Avoid: fragmentation of memory
(since each malloc needs to give consecutive bytes)

malloc: select smallest free chunk to satisfy a request

free: coalesces contiguous free chunks (like mercury)

Portability across OS's by using malloc, free.

Savithri Sathyvan story: stack growth